

Métodos Analíticos y Programación

Trabajo Práctico 3

Escuela de Negocios, UTDT

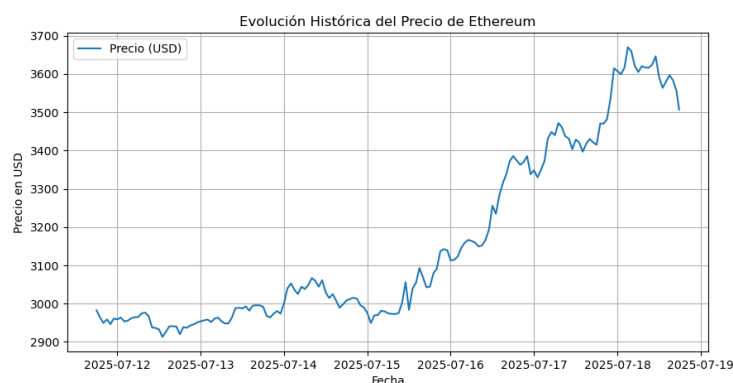
Segundo Semestre de 2025

Observaciones generales:

- El trabajo se debe realizar en grupos de 3 personas.
- El código fuente debe estar bien comentado. Cualquier aclaración adicional que se considere necesaria debe ser incluida como comentarios en el código fuente.
- La resolución del TP debe subirse al formulario *TP3: Entrega* en la página de la materia en el campus virtual.
- La fecha límite de entrega es el martes **4 de noviembre a las 23:55**. Se aceptarán TPs hasta 48 horas pasado el deadline, pero en ese caso el rango [60-100] se reescalará al rango [60-80].

CoinGecko es una plataforma líder de análisis y seguimiento de datos de criptomonedas. Fundada en 2014, proporciona una amplia variedad de información sobre criptomonedas y mercados, incluyendo precios, capitalización de mercado, volúmenes de intercambio y métricas más avanzadas. Su objetivo es democratizar el acceso a datos crypto, ayudando a inversores, desarrolladores y entusiastas a tomar decisiones informadas.

El objetivo de este trabajo es crear un programa en Python que grafique la evolución del valor de algunas criptomonedas a lo largo del tiempo. En particular, servirá para hacer un seguimiento de los valores de las crypto Bitcoin, Ethereum, Tether y Solana. A continuación puede observarse una imagen que ilustra el tipo de gráfico que se espera que genere el programa:



Al ejecutar el programa, en primer lugar se deberá desplegar un menú para elegir la criptomoneda del gráfico a generar:

```
Criptomonedas
-----
1. Bitcoin
2. Ethereum
3. Tether
4. Solana
Seleccione una criptomoneda:
```

Luego debe pedírsele al usuario que indique la cantidad de días que se deben tener en cuenta para realizar el gráfico (contabilizando de la fecha actual hacia atrás):

```
Ingrese la cantidad de días a visualizar:
```

En función de la criptomoneda y la cantidad de días seleccionados, deberá generarse el gráfico correspondiente.

Para obtener la información necesaria, *CoinGecko* dispone una API que puede ser accedida mediante requerimientos HTTP. En particular, puede consultarse el valor de una criptomoneda (que se identifica con un código) durante un período de tiempo. Para accederla, se debe realizar una consulta considerando lo siguiente:

- **Endpoint:** `https://api.coingecko.com/api/v3/coins/cripto_id/market_chart` (reemplazando *cripto_id* por el código de la criptomoneda que se desea consultar).
- **Parámetros:**
 - `vs_currency` (str): código de la moneda de los valores que se obtienen en la respuesta (por simplicidad, en este trabajo fijaremos el valor del parámetro en `usd`).
 - `days` (int): la cantidad de días por los que se está consultando, desde el momento en que se hace la consulta hacia atrás. (Por ejemplo, 10 indica que se consulta el valor de la criptomoneda durante los últimos 10 días)

Los códigos de las criptomonedas que pueden consultarse son `'bitcoin'`, `'ethereum'`, `'tether'` y `'solana'`.

Como respuesta a un requerimiento, la API de *CoinGecko* devuelve un JSON que tiene la siguiente estructura:

```
{
  "prices": [[1731890441562, 90036.20358450076],
             [1731890735905, 90076.18898792121], ...],
  "market_caps": ...,
  "total_volumes": ...
}
```

Para generar los gráficos, sólo debe tenerse en cuenta los valores de las criptomonedas a lo largo del tiempo, información que se encuentra bajo la clave `'prices'`. El valor asociado a esta clave es un `Array` que dentro tiene `Arrays`, con dos números cada uno: el primero es un *timestamp* (un instante en el tiempo) y el segundo el valor de la criptomoneda en ese instante.

Un *timestamp* (por ejemplo, 1731890441562) es una marca de tiempo, que debe interpretarse como la cantidad de milisegundos que ocurrieron desde el 1 de enero de 1970. Para incluirlo en el gráfico, hay que convertirlo a un formato de fecha legible. Esto puede hacerse importando la biblioteca `datetime`. Por ejemplo, el valor obtenido al hacer

```
datetime.datetime.fromtimestamp(1731890441562/1000)
```

puede incorporarse al gráfico sin inconvenientes. (Notar que en este caso el valor del *timestamp* se está dividiendo por 1000, para descartar milisegundos).

La carpeta adjunta `templates` contiene un archivo de ejemplo para usar como referencia. En particular, se deben implementar las funciones `obtener_datos_cripto(...)` y `graficar_cripto(...)` del archivo `tp3.py`, incluyendo además todas las funciones auxiliares que se consideren necesarias.

A continuación se enumeran algunas funciones de `matplotlib` que pueden resultar útiles para generar los gráficos requeridos:

- `plt.grid(b)`: establece si debe mostrarse una grilla sobre el gráfico (`b` es `bool`).
- `plt.title(título)`: establece el título del gráfico (`título` es `str`).
- `plt.xlabel(etiqueta)`: establece el texto junto el eje X (`etiqueta` es `str`).
- `plt.ylabel(etiqueta)`: establece el texto junto el eje Y (`etiqueta` es `str`).